Unlike the Ising model examples, we're going to create separate functions to select actions for each agent and to do training, since this is a more complex environment and we want to modularize a bit more. After each step in the environment, we get an observation tensor for all the agents simultaneously.

The observation returned by `env.get_observation(team1)` is actually a tuple with two tensors. The first tensor is shown in the top portion of figure 9.26. It is a complex high-order tensor, whereas the second tensor in the tuple has some additional information that we will ignore. From now on, when we say *observation* or *state*, we mean the first tensor as depicted in figure 9.26.

Figure 9.26. The structure of the observation tensor. It is an *N* x 13 x 13 x 7 tensor where *N* is the number of agents on the team.

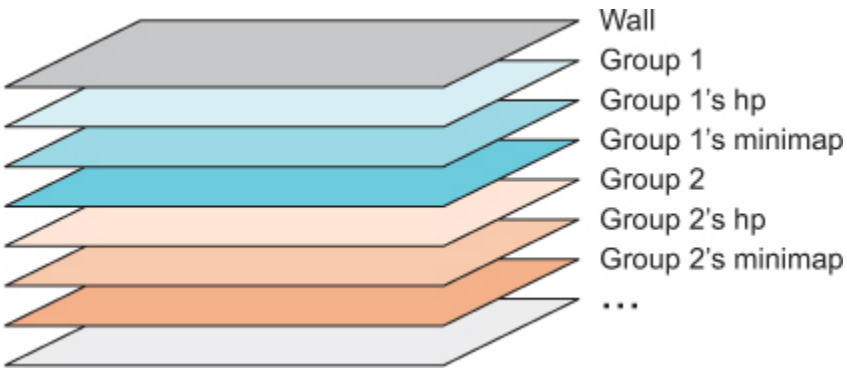


Figure 9.26 shows that this observation tensor is arranged in slices. The observation is an $N \times 13 \times 13 \times 7$ tensor where N is the number of agents (in our case 16). Each 13×13 slice of the tensor for a single agent shows the FOV with the location of the wall (slice 0), team 1 agents (slice 1), team 1 agents' HPs (slice 2), and so forth. We will only be using slices 1 and 4 for the locations of the agents on team 1 and team 2 within the FOV. So, a single agent's observation tensor will be $13 \times 13 \times 2$, and we will flatten this into a vector to get a 338-length state vector. We'll then concatenate this state vector with the mean field vector, which is length 21, to get a $338 + 21 = 359$ length vector that will be given to the Q function. It would be ideal to use a two-headed neural network like we did in chapter 7. That way one head could process the state vector and the other could process the mean field action vector, and we could then recombine the processed